

Trabajo de Campo Integrador

Materia: Ingeniería de Software II

Cuatrimestre: 2026 — IUA

Modalidad: Grupal (3-4 personas)

Duración: 7 semanas (29/04 al 16/06)

Peso en la calificación final: consultar régimen vigente

Objetivo

Aplicar los conceptos vistos a lo largo de toda la materia (Temas I a VIII) sobre un **proyecto de software propio**, desde su concepción hasta su evaluación. Al finalizar el trabajo, cada grupo debe poder demostrar:

- Un sistema de software funcional con calidad medible
 - Un plan de calidad (SQA) que documente cómo se aseguró la calidad durante el desarrollo
 - Una matriz de trazabilidad (RTM) que vincule requerimientos, código y tests
 - Métricas de código y reportes de pruebas
 - Wireframes/prototipos del frontend
 - Defensa oral del proyecto
-

Conformación de grupos

- **Tamaño:** 3 a 4 integrantes (excepciones consultar con docente).
 - **Cuándo:** se conforman en la clase del **miércoles 29/04** (Sem 7).
 - **Responsabilidades:** cada grupo designa un **referente** que coordina entregas con el docente.
-

Tipos de proyectos sugeridos

Cualquiera de los siguientes (el grupo puede proponer otro a aprobación del docente):

Tipo	Ejemplos
Sistema de gestión	Turnos médicos, biblioteca, inventario, reservas, gimnasio
Aplicación de utilidad	Lista de tareas, finanzas personales, tracker de hábitos
API + frontend simple	API REST con CRUD + interfaz web mínima
Plataforma básica	Blog, foro, wiki, marketplace de barrio
Herramienta de productividad	Generador de informes, conversor, calculadora especializada

Recomendación: elegir un proyecto **acotado pero realista**. Mejor terminar bien algo chico que tener algo grande sin terminar.

Restricciones de alcance

- **LOC objetivo:** entre 800 y 3000 líneas de código (sin contar dependencias).
- **Funcionalidades mínimas:** al menos 5 requerimientos funcionales documentados.
- **Persistencia:** debe haber alguna forma de almacenamiento (base de datos, archivos, API externa).
- **Stack tecnológico:** libre (Python, JavaScript/TypeScript, Java, Go, Ruby, etc.). El docente debe poder ejecutar el proyecto siguiendo el README.

Hitos y entregas

#	Hito	Semana	Fecha	Entregable	Peso
1	Conformación de grupos + idea preliminar	7	29/04	Lista de integrantes + 3 ideas posibles de proyecto	—
2	Propuesta de proyecto definida	8	05/05	Documento de 1 página por mail antes del martes 19:00 + mini-presentación 3 min en clase	10%
3	Avance 1: Métricas + Plan SQA	10	20/05	Plan SQA + métricas iniciales + linter funcionando	20%
4	Avance 2: Pruebas + gestión de defectos	12	03/06	Plan de pruebas + casos de prueba + reporte de defectos	25%
5	Entrega final + Defensa oral	14	16/06	Proyecto completo + RTM + wireframes + documentación + defensa	45%

Política de entregas: cada entrega se hace **al inicio de la clase de la fecha indicada**. Las entregas tardías se descuentan 10% por día.

Detalle de entregables por hito

Hito 1 — Sem 7 (29/04): Conformación de grupos + idea preliminar

Qué se entrega:

- Nombre del grupo (puede ser informal)
- Lista de integrantes (nombre, apellido, email)
- 3 ideas posibles de proyecto (1-2 líneas cada una)

Cómo se entrega: verbalmente al docente al final de la clase del miércoles 29/04. No requiere documento escrito.

Hito 2 — Sem 8 (05/05): Propuesta de proyecto

Qué se entrega: documento PDF de **1 página** con la siguiente estructura:

PROPUESTA DE PROYECTO

Grupo: _____

Integrantes: _____

1. NOMBRE DEL PROYECTO: _____

2. PROBLEMA QUE RESUELVE (3-5 líneas):

3. USUARIOS / PÚBLICO OBJETIVO: _____

4. ALCANCE FUNCIONAL – al menos 5 requerimientos:

REQ-F01: _____

REQ-F02: _____

REQ-F03: _____

REQ-F04: _____

REQ-F05: _____

5. ALCANCE NO FUNCIONAL – al menos 2 requerimientos:

REQ-NF01: _____

REQ-NF02: _____

6. STACK TECNOLÓGICO:

- Lenguaje: _____

- Framework: _____

- Base de datos: _____

- Otras herramientas: _____

7. REPOSITORIO (URL pública en GitHub/GitLab): _____

8. RIESGOS PRINCIPALES Y CÓMO LOS MITIGARÁN (2-3 ítems):

- _____

- _____

Forma de entrega: PDF por mail al docente **antes del martes 5/5 a las 19:00** + presentación oral de 3 min en clase del martes 5/5 (después del simulacro).

Criterios de aprobación: alcance realista, requerimientos verificables, stack viable.

Hito 3 — Sem 10 (20/05): Avance 1 — Métricas + Plan SQA

Qué se entrega:

1. **Plan SQA simplificado** (basado en IEEE 730), con al menos estas secciones:
2. Propósito del plan
3. Roles del equipo (quién hace QA, quién hace QC, quién es referente)
4. Estándares y convenciones (estilo de código, naming, etc.)
5. Estrategia de revisiones (code review en cada PR, etc.)

6. Métricas a monitorear y umbrales (CC, MI, cobertura)
7. Herramientas (linter, formateador, framework de tests)
8. **Métricas iniciales del código:**
9. Líneas de código (LOC)
10. Complejidad ciclomática (CC) por módulo (con `radon cc -s` para Python o equivalente)
11. Maintainability Index (MI) global
12. Reporte de linter (sin errores críticos)
13. **Estado del repositorio:**
14. URL del repo público
15. Al menos 10 commits con mensajes descriptivos
16. README con instrucciones de instalación
17. Linter configurado y ejecutándose

Forma de entrega: carpeta ZIP con: documento PDF del Plan SQA + reporte de métricas + link al repo.

Hito 4 — Sem 12 (03/06): Avance 2 — Pruebas + Gestión de defectos

Qué se entrega:

1. **Plan de pruebas:**
2. Alcance (qué se va a testear, qué no)
3. Estrategia (caja negra, caja blanca, integración)
4. Criterio de aceptación (cobertura mínima esperada, etc.)
5. **Set de casos de prueba** (al menos 15 casos):
6. Casos de caja negra (partición de equivalencia, valores límite)
7. Casos de caja blanca (cobertura de sentencias, ramas)
8. Cada caso con: ID, requerimiento asociado, precondition, pasos, resultado esperado, resultado real
9. **Implementación de los tests:**
10. Tests automatizados ejecutándose (pytest, jest, junit, etc.)
11. Cobertura $\geq 60\%$ sobre los módulos principales
12. **Reporte de defectos:**
13. Al menos 5 defectos detectados durante el desarrollo
14. Cada defecto con: ID, severidad, descripción, paso para reproducir, estado (abierto/resuelto)

Forma de entrega: carpeta ZIP con: documento del plan + planilla de casos de prueba + reporte de defectos + reporte de cobertura + link al repo actualizado.

Hito 5 — Sem 14 (16/06): Entrega final + Defensa oral

Qué se entrega:

1. **Proyecto completo** en repositorio público con:
2. Código funcional y ejecutable
3. README completo (instalación, uso, ejemplos)
4. Suite de tests pasando
5. Pipeline de calidad documentado (linter + tests + métricas)
6. **Documentación final:**
7. **Catálogo de requerimientos** (REQ-F01..., REQ-NF01...) con ID, descripción, prioridad, fuente
8. **Matriz de Trazabilidad de Requerimientos (RTM)** completa: requerimiento → módulo/función → test → estado
9. **Plan SQA** actualizado (versión final del Hito 3)
10. **Plan de pruebas + reporte final** (versión actualizada del Hito 4)
11. **Métricas finales:** LOC, CC, MI, cobertura de tests
12. **Wireframes / prototipos** (Figma, Balsamiq o similar). Mínimo 3 pantallas en wireframe + 2 en alta fidelidad
13. **Reflexión final:** 1 página sobre qué aprendieron, qué harían distinto, qué temas de la materia aplicaron
14. **Defensa oral** en la clase del 16/06:
15. Duración: 10-15 min por grupo
16. Demo en vivo del sistema funcionando
17. Cada integrante presenta al menos una sección
18. Preguntas del docente y compañeros

Forma de entrega: repositorio actualizado + carpeta ZIP con toda la documentación + presentación oral.

Criterios de evaluación

Criterio	Peso	Qué se evalúa
Aplicación de conceptos de la materia	40%	Uso correcto de RTM, ISO 25010, SQA, métricas, pruebas, UX
Calidad del código y trazabilidad	30%	Código limpio, modular, con trazabilidad embebida (Trazabilidad: REQ-XXX)
Pruebas y métricas	20%	Cobertura $\geq$ 60%, casos relevantes, defectos documentados
Presentación y documentación	10%	Documentación clara, defensa oral preparada, wireframes legibles

Rúbrica por hito

Nivel	Descripción
Excelente (10)	Cumple todo lo solicitado, agrega valor extra, documentación clara, trazabilidad completa
Bueno (7-8)	Cumple lo principal, algunos detalles menores faltantes, trazabilidad parcial
Aprobado (4-6)	Cumple lo mínimo, varios huecos, requiere mejoras significativas
Insuficiente (1-3)	No cumple lo mínimo, faltan entregables principales

Reglas operativas

1. **Repositorio público obligatorio** en GitHub o GitLab desde el Hito 2. El docente debe tener acceso de lectura.
2. **Commits regulares:** cada integrante debe tener commits propios. No se acepta "todo el código en un commit final".
3. **Mensajes de commit descriptivos:** evitar `fix` , `update` , `cambios` . Usar formato `[REQ-F01] Implementa registro de pacientes` .
4. **README completo:** instalación, requisitos previos, comandos para ejecutar tests y el sistema.
5. **No se permite código copiado** sin atribución. Las dependencias externas (librerías) están permitidas pero deben declararse.
6. **Reuniones de seguimiento:** el docente puede solicitar reuniones de 15 min con cada grupo entre hitos.

Calendario completo

Semana	Fecha	Hito	Actividad
7	29/04	1	Conformación de grupos + idea preliminar (mié)
8	05/05	2	Entrega de propuesta (mar 19:00 por mail) + mini-presentación (mar)
8	06/05	—	PARCIAL 1 (mié, escrito individual, Temas I-IV)
9	12-13/05	—	Tema V Medición — aplicar a su proyecto
10	20/05	3	Avance 1: Métricas + Plan SQA (mié)
11	26-27/05	—	Tema VII Pruebas parte 1 — diseñar casos para su proyecto
12	03/06	4	Avance 2: Pruebas + defectos (mié)
13	09-10/06	—	Tema VIII UX — diseñar wireframes de su proyecto
14	16/06	5	Entrega final + Defensa oral (mar)

Tips de un docente que ya vio muchos trabajos de campo

1. **Empiecen ya.** Las primeras 2 semanas se sienten lejanas, pero son las que más rinden.
2. **Documenten desde el primer commit.** Trazabilidad embebida es 10x más fácil que reconstruirla al final.
3. **No se enamoren del alcance.** Recortar es difícil pero necesario. Mejor 5 funcionalidades pulidas que 15 a medias.
4. **Hagan code review entre ustedes.** Es práctica real de Tema II y mejora la calidad sin esfuerzo extra.
5. **Tests primero, defectos después.** Si encuentran defectos sin tests asociados, agreguen el test antes de arreglar.
6. **El parcial 2 cubre Temas V-VIII. El trabajo de campo aplica TODO. Estudien con su propio proyecto.**

Contacto

Cualquier duda, consultar al docente por mail o en clase. Las dudas frecuentes se publican como FAQ.

¡Éxitos en el proyecto!